

Análisis de Discursos Políticos de la Campaña Presidencial de Estados Unidos 2020

Tarea 2 - Introducción a la Ciencia de Datos - FING 2025

Ignacio Campón y Mauro Loprete

Introducción

Este análisis explora 269 discursos políticos recogidos durante la campaña presidencial de Estados Unidos en 2020. Se han seleccionado los cinco oradores con mayor cantidad de intervenciones (Joe Biden, Donald Trump, Mike Pence, Bernie Sanders y Kamala Harris) tras confirmar, mediante una revisión automatizada y manual, que los datos son lo suficientemente completos y homogéneos para un estudio riguroso de patrones de publicación y análisis textual.

La base de datos incluye seis variables: tres categóricas (*speaker*, *type*, *location*), dos de texto (*title*, *text*) y una temporal (*date*). Los registros comprenden intervenciones realizadas entre el 15 de enero y el 16 de octubre de 2020. Para preparar el corpus se implementó una función de limpieza que:

1. Elimina la introducción redundante en el texto, suprimiendo el contenido previo al primer salto de línea.
2. Convierte todo el texto a minúsculas para evitar duplicaciones por diferencias de mayúsculas.
3. Elimina marcas de tiempo (por ejemplo, (00:01) o (1:05:12)) comunes en transcripciones automáticas.
4. Suprime encabezados de oradores (como “joe biden:” o “president trump”) para evitar distorsiones en el conteo de palabras.
5. Quita signos de puntuación innecesarios y espacios adicionales.

Esta cadena de transformaciones permite obtener un vocabulario depurado y coherente que resulta indispensable para análisis posteriores como el estudio de frecuencias, sentimientos o co-ocurrencias de términos.

La Figura 1 muestra la frecuencia de discursos por candidato en la campaña presidencial de Estados Unidos 2020. Se observa que Joe Biden y Donald Trump son los candidatos con

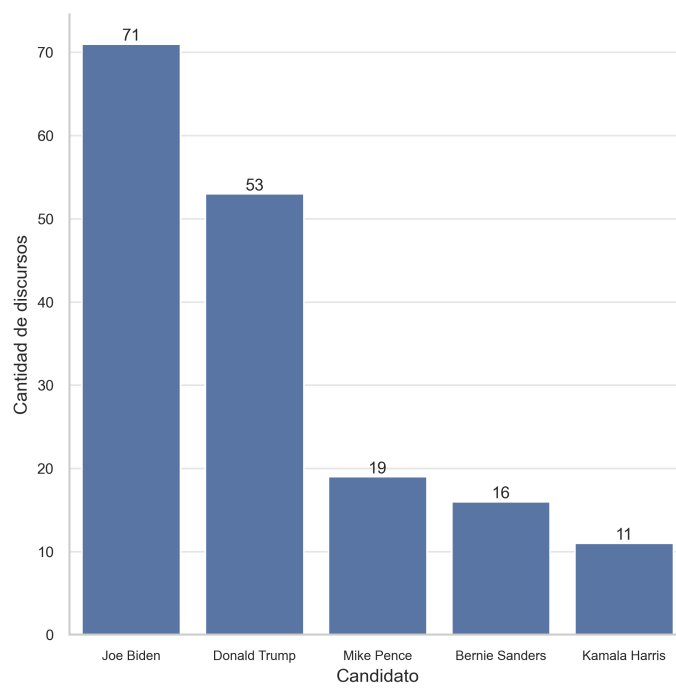


Figura 1: Frecuencia de discursos por candidato (Campaña 2020).

mayor número de discursos, seguidos por Mike Pence, Bernie Sanders y Kamala Harris. Esta distribución refleja la actividad política y mediática de cada candidato durante la campaña.

Tras seleccionar los tres candidatos con más discursos: Joe Biden, Donald Trump y Mike Pence, se procede a dividir el conjunto de datos en dos partes: un conjunto de desarrollo (entrenamiento) y un conjunto de prueba (test). Esta división nos permite evaluar el rendimiento de los modelos de clasificación en datos no vistos, asegurando que las métricas obtenidas sean representativas del desempeño real del modelo. Para construir estos conjuntos, se utiliza un muestreo estratificado, garantizando que la proporción de discursos de cada candidato se mantenga en ambos conjuntos. Esto es crucial para evitar sesgos en el modelo y asegurar una evaluación justa. Obsérvese la Figura 2, que muestra la distribución de discursos por candidato en los conjuntos de desarrollo y prueba, confirmando que la proporción de discursos se mantiene constante entre ambos conjuntos.

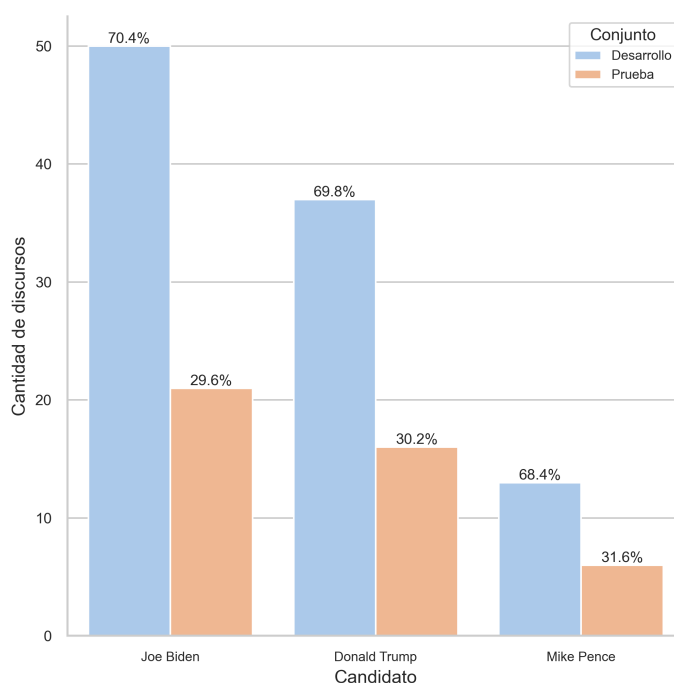


Figura 2: Distribución de discursos por candidato y conjunto.

Para transformar el texto del conjunto de entrenamiento a una representación numérica se utilizó la técnica de Bag of Words (BoW), implementada a través del `CountVectorizer` de `scikit-learn`. Esta técnica convierte cada documento en un vector donde cada componente representa la frecuencia absoluta de una palabra específica del vocabulario del corpus.

BoW parte del supuesto de que el orden de las palabras no importa. Se construye una matriz $D \times V$, donde, 4 es la cantidad de discursos (filas), V es el tamaño del vocabulario total

(columnas), y cada celda $[i, j]$ contiene el número de veces que la palabra j aparece en el documento i .

En otras palabras, cada documento se representa como un vector de conteo de palabras, sin importar su contexto o posición.

Se transforma el conjunto de entrenamiento de dos maneras:

- a. Unigramas incluyendo stopwords No se eliminan palabras vacías (stopwords), por lo que el vocabulario es más amplio pero contiene muchas palabras con poca capacidad discriminativa. Entre los 100 discursos existen 13.704 palabras únicas (unigramas) en el corpus. La matriz resultante tiene una proporción de entradas no nulas: ~ 0.0889 .
- b. Unigramas + bigramas, excluyendo stopwords en inglés Se remueven stopwords usando la lista estándar del idioma inglés y se incluyen también combinaciones de dos palabras (bigramas), lo cual expande el tamaño del vocabulario pero aporta información contextual más rica. La dimensión de la matriz resulta en los 100 discursos, 197.501 combinaciones de bigramas — es decir, un vocabulario muchísimo más grande al incorporar combinaciones de dos palabras. La matriz resultante tiene una proporción de entradas no nulas: ~ 0.0197 .

A pesar del alto número de columnas (features), cada documento contiene solo un pequeño subconjunto del vocabulario total, por lo que la mayoría de las entradas de la matriz son ceros. Por esta razón, la matriz resultante es una matriz dispersa y para ahorrar memoria y acelerar los cálculos son almacenadas guardando únicamente las entradas no nulas. De hecho, en el caso del vectorizador con unigramas y bigramas, la matriz tiene casi 200 mil columnas, pero menos del 2% de sus celdas contienen información distinta de cero. En el primer caso, con unigramas y stopwords, la matriz tiene más de 13 mil columnas, pero menos del 9% de sus celdas contienen información distinta de cero.

Este comportamiento es típico en tareas de procesamiento de lenguaje natural. Si se trabajara con millones de documentos, representar esta matriz como densa (con todos los ceros explícitos) consumiría cantidades de memoria astronómicas e inviables para la mayoría de los equipos. Por ello, se emplean estructuras de datos especializadas que almacenan únicamente las posiciones y valores de las entradas no nulas, permitiendo un uso eficiente de la memoria y un procesamiento más rápido. Esta técnica es sencilla y efectiva, pero produce vectores de alta dimensionalidad y dispersos, lo que hace fundamental el uso de representaciones dispersas para su manejo computacional.

Una mejor representación que Bag of Words, se puede utilizar TF-IDF (Term Frequency-Inverse Document Frequency), que pondera la importancia de cada palabra en el contexto del corpus, reduciendo el impacto de palabras comunes y mejorando la capacidad discriminativa del modelo. Un ejemplo de la representación de TF-IDF se muestra a continuación:

- Documento 1: “el gato come pescado”
- Documento 2: “el perro come carne”

El vocabulario es: [“el”, “gato”, “come”, “pescado”, “perro”, “carne”]

Para cada término, calculamos:

- TF (frecuencia de término): número de veces que aparece la palabra en el documento.
- DF (document frequency): número de documentos donde aparece la palabra.
- IDF (inversa de la frecuencia de documento):

$$\text{IDF}(t) = \log \left(\frac{N}{\text{DF}(t)} \right)$$

donde N es el número total de documentos.

Por ejemplo, para la palabra “gato”:

- TF en Documento 1: 1, en Documento 2: 0
- DF: 1 (solo aparece en Documento 1)
- IDF: $\log(2/1) = 0.301$

Para la palabra “el”:

- TF en ambos documentos: 1
- DF: 2 (aparece en ambos)
- IDF: $\log(2/2) = 0$

El vector TF-IDF para **Documento 1** sería:

- [“el”: 0, “gato”: 0.301, “come”: 0, “pescado”: 0.301, “perro”: 0, “carne”: 0]

Y para Documento 2:

- [“el”: 0, “gato”: 0, “come”: 0, “pescado”: 0, “perro”: 0.301, “carne”: 0.301]

Modelo de aprendizaje

Una vez obtenida la representación numérica de los discursos mediante la transformación TF-IDF, se procede a entrenar un modelo de aprendizaje supervisado para clasificar los discursos según el candidato que los pronunció.

En este caso se utiliza el modelo **Naive Bayes Multinomial**, ampliamente utilizado en tareas de clasificación de texto por su simplicidad, rapidez y buenos resultados incluso con datos ruidosos o de alta dimensión.

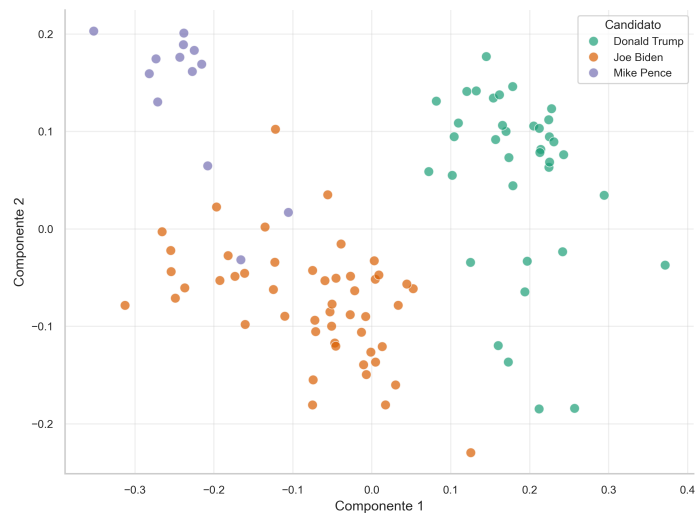


Figura 3: Proyección PCA sobre el conjunto de desarrollo (TF-IDF 1).

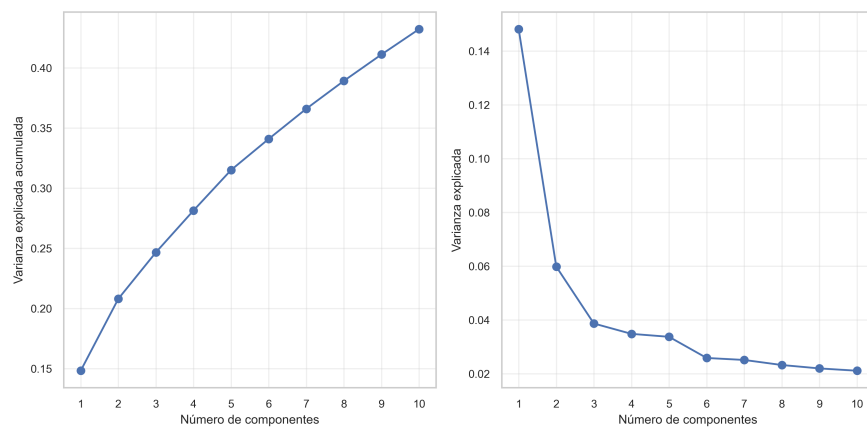


Figura 4: Varianza explicada acumulada y por componente (TF-IDF 1).

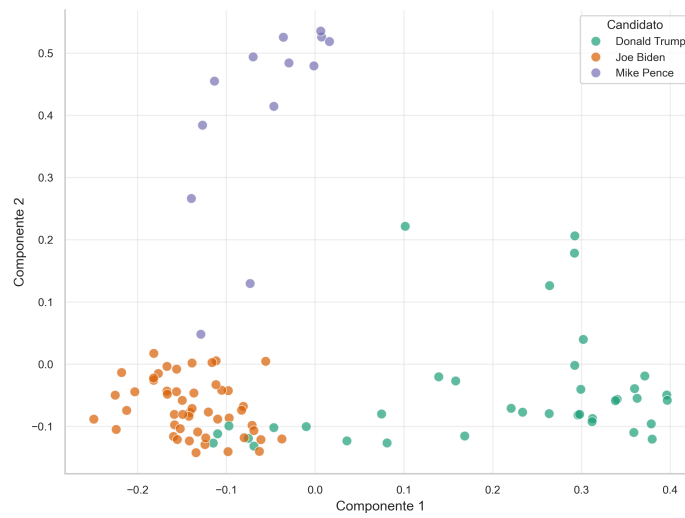


Figura 5: Proyección PCA sobre el conjunto de desarrollo (TF-IDF 2).

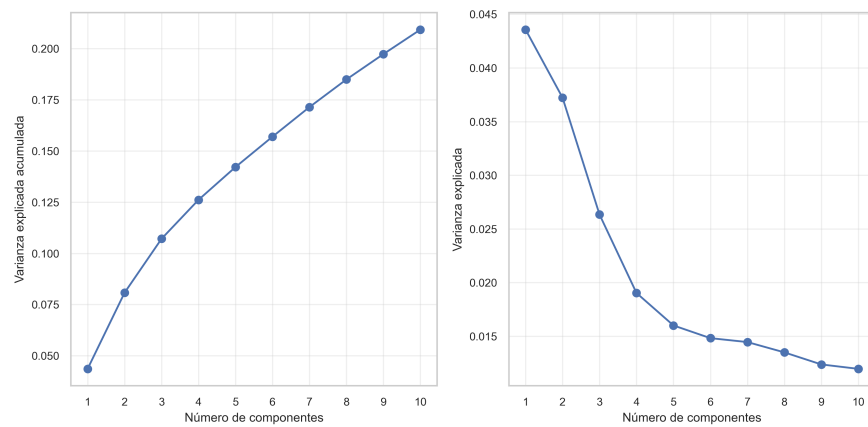


Figura 6: Varianza explicada acumulada y por componente (TF-IDF 2).

Naive Bayes Multinomial

El modelo se basa en el **Teorema de Bayes** (?), que permite estimar la probabilidad de una clase c dado un documento d :

$$P(c | d) = \frac{P(c) \cdot P(d | c)}{P(d)}$$

Como $P(d)$ es constante para todas las clases, el modelo predice la clase que maximiza la siguiente expresión:

$$\hat{c} = \arg \max_c (P(c) \cdot P(d | c))$$

Para documentos representados como vectores de frecuencia de términos $\mathbf{x} = (x_1, x_2, \dots, x_n)$, donde x_i es el número de veces que aparece el término i , el modelo **multinomial** asume que:

$$P(d | c) = \prod_{i=1}^n P(w_i | c)^{x_i}$$

Donde $P(w_i | c)$ es la probabilidad de observar la palabra w_i en un documento de la clase c . Esta probabilidad se estima mediante:

$$P(w_i | c) = \frac{N_{ic} + \alpha}{N_c + \alpha V}$$

- N_{ic} : número total de ocurrencias del término w_i en los documentos de la clase c
- N_c : número total de palabras en los documentos de la clase c
- V : tamaño del vocabulario
- α : parámetro de suavizado

La predicción final se realiza entonces con:

$$\hat{c} = \arg \max_c \left(\log P(c) + \sum_{i=1}^n x_i \log P(w_i | c) \right)$$

Métricas de evaluación

Para evaluar el desempeño del modelo entrenado se utilizan las siguientes métricas:

- **Accuracy:** proporción de predicciones correctas sobre el total de observaciones del conjunto de prueba:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(y_i = \hat{y}_i)$$

- **Matriz de confusión:** tabla que muestra el número de predicciones correctas e incorrectas para cada clase, permitiendo identificar patrones de error.
- **Precisión (precision):** proporción de verdaderos positivos sobre el total de predicciones positivas para una clase:

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}$$

- **Recall (sensibilidad):** proporción de verdaderos positivos sobre el total de instancias reales de la clase:

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}$$

En la tabla Figura 7 se presentan los resultados del modelo aplicados al conjunto de test, utilizando las métricas mencionadas, cabe mencionar que nos encontramos con un problema importante para clasificar los discursos de los candidatos ya que si consideramos las predicciones del conjunto de test todas son para Biden, esto se puede ver mas en detalle en la matriz de confusión mencionada posteriormente.

Candidato	Precisión	Recall
Joe Biden	0.49	1.00
Donald Trump	0.00	0.00
Mike Pence	0.00	0.00

Figura 7: Métricas de precisión y recall del modelo Naive Bayes por candidato en el conjunto de test.

El **accuracy** del modelo sobre el conjunto de test resulta en un bajo valor de 0.49. En la Figura 8, se presenta la matriz de confusión, que muestra el número de aciertos y errores cometidos por el modelo al clasificar a cada candidato. Esta visualización permite identificar

patrones sistemáticos de error, particularmente en la clasificación de los discursos de *Donald Trump* y *Mike Pence*, quienes no fueron correctamente identificados en ningún caso. Esto evidencia una limitada capacidad del modelo para distinguir entre ciertos oradores, lo que compromete su rendimiento en escenarios multiclase con discursos de estilos similares.

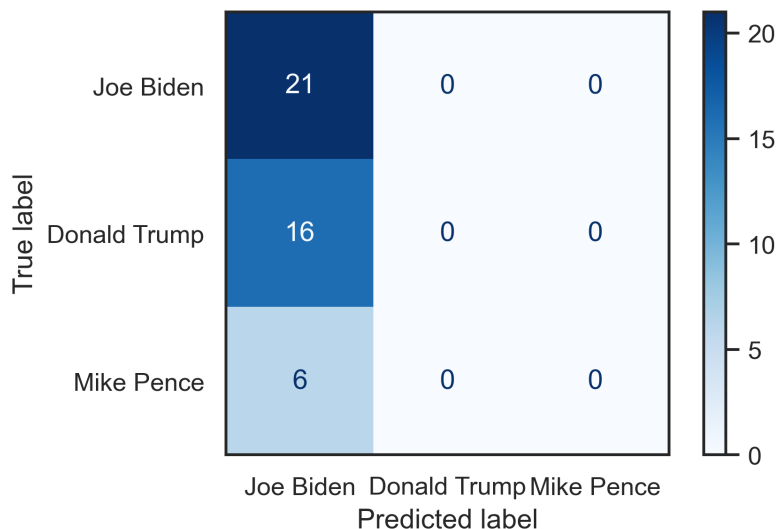


Figura 8: Matriz de Confusión Naive Bayes.

Validación Cruzada y Búsqueda de Hiperparámetros

La validación cruzada (cross-validation) es una técnica que evalúa el desempeño de un modelo dividiendo el conjunto de datos en k subconjuntos o folds. En cada iteración, el modelo se entrena en $k - 1$ folds y se evalúa en el fold restante. Al promediar las métricas obtenidas en las k iteraciones se obtiene una estimación robusta del rendimiento, evitando así el sobreajuste provocado por depender de un único conjunto de validación.

Para optimizar los hiperparámetros usamos GridSearchCV, el cual explora sistemáticamente todas las combinaciones de una cuadrícula de parámetros predefinida. En cada iteración de la validación cruzada se evalúa una combinación determinada, y se puede visualizar la distribución del accuracy (u otra métrica) mediante un gráfico de violín que muestre el valor promedio y la variabilidad en cada uno de los splits.

La búsqueda de hiperparámetros es crucial para mejorar el rendimiento del modelo, ya que permite ajustar parámetros como la suavización (α) y la inclusión de priors, que pueden influir significativamente en la capacidad del modelo para generalizar a nuevos datos. La cuadrícula definida para la búsqueda de hiperparámetros incluye valores de α que van desde 0.001 hasta 5.0, y la opción de ajustar el parámetro `fit_prior` para considerar o no las probabilidades a priori de las clases.

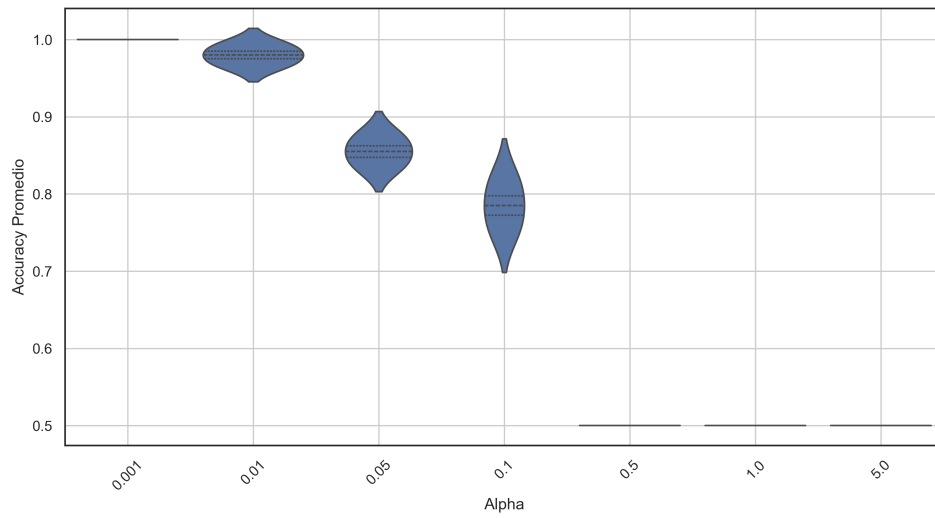


Figura 9: Comparación de Accuracy por Parámetro Alpha del Cross Validation.

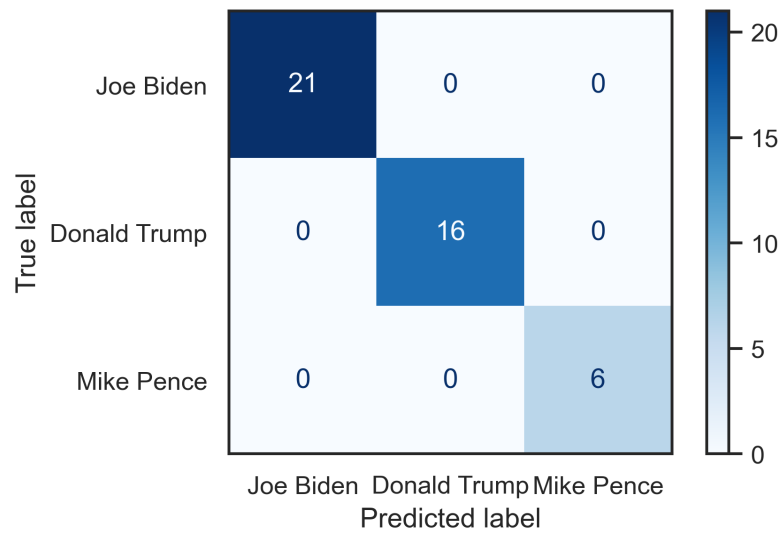


Figura 10: Matriz de Confusión del Naive Bayes bajo optimización de Hiperparámetros.

Selección del Mejor Modelo y Retrenamiento Completo

Una vez identificada la mejor combinación de hiperparámetros, se reentrena el modelo usando todo el conjunto de entrenamiento (sin apartar datos para validación). Se reportan las métricas finales, como accuracy, precision y recall, y se presenta la matriz de confusión. Es importante señalar que los modelos basados en representaciones bag-of-words o TF-IDF tienen limitaciones: ignoran el orden y contexto de las palabras y generan vectores de alta dimensionalidad y dispersos, lo que puede dificultar capturar relaciones semánticas complejas.

Modelo Alternativo: Regresión Logística

Además de Naive Bayes, se evaluó Regresión Logística, un modelo lineal que estima probabilidades usando softmax sobre combinaciones lineales de features.

Regresión Logística

La Regresión Logística es un modelo lineal que estima las probabilidades de pertenencia a cada clase aplicando la función softmax sobre una combinación lineal de las features. Para cada clase c , se calcula:

$$P(c \mid \mathbf{x}) = \frac{e^{\mathbf{w}_c^\top \mathbf{x} + b_c}}{\sum_k e^{\mathbf{w}_k^\top \mathbf{x} + b_k}}$$

donde $\mathbf{w}_c$ y b_c son los parámetros asociados a la clase c . Dicho modelo se entrena maximizando la verosimilitud, usualmente a través de métodos numéricos como el algoritmo de Newton-Raphson (NR), el cual ajusta iterativamente los pesos y el sesgo para maximizar la función de verosimilitud y acercar las probabilidades predichas a las etiquetas reales. A diferencia de Naive Bayes, que estima $P(x \mid c)$ y aplica la regla de Bayes, la Regresión Logística modela directamente la probabilidad condicional $P(c \mid \mathbf{x})$, permitiendo capturar interacciones lineales entre las variables.

En este modelo existen dos hiperparámetros principales:

- **C**: inverso de la regularización, controla la complejidad del modelo. Valores más altos permiten mayor flexibilidad, mientras que valores bajos penalizan más las magnitudes de los coeficientes.
- **penalty**: tipo de regularización a aplicar (L1, L2 o ninguna). La regularización L1 tiende a generar modelos más esparsos, mientras que L2 penaliza las magnitudes de los coeficientes sin forzarlos a cero.

Se define una cuadrícula de valores de C en $[0.01, 0.1, 1, 10]$ en conjunto con las penalizaciones L1, L2 y elasticnet. Se utiliza el método de validación cruzada con 10 folds para evaluar el rendimiento del modelo en cada combinación de hiperparámetros. Posteriormente, se entrena el modelo con los mejores parámetros encontrados ($C = 10$ y $penalty = l2$) y se evalúa su rendimiento en el conjunto de test resultando en un accuracy del 98% donde el modelo logra clasificar correctamente todos los discursos de los candidatos a excepción de uno de Biden que lo confundió con Pence.

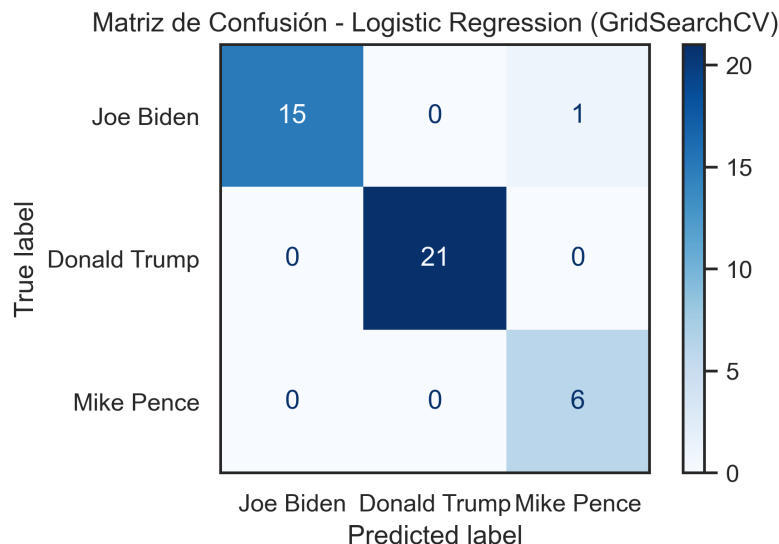


Figura 11: Matriz de Confusión del modelo de Regresión Logística.

Consideraciones sobre (Des)balance y Extracción Alternativa de Features

Al cambiar la selección de candidatos es posible observar problemas de balance en los datos, situación que puede sesgar las predicciones. Técnicas como el sobre-muestreo y el submuestreo son herramientas útiles para rebalancear la distribución de clases y mitigar este efecto, aunque no se implemente en este caso.

Una consideración adicional es que, en Python, la librería imbalanced-learn implementa diversas técnicas de remuestreo—entre ellas SMOTE y otras variantes—lo que facilita su integración en pipelines de scikit-learn para manejar el desbalance de clases.

Alternativas de Extracción de Features

En el ámbito del procesamiento de lenguaje natural, existen diversas técnicas para representar texto como vectores numéricos. A continuación se describen algunas de las más relevantes:

Word2Vec

Word2Vec (Mikolov et al., 2013) aprende vectores densos de palabras mediante modelos de Continuous Bag-of-Words (CBOW) o Skip-Gram, optimizando la predicción de palabras y su contexto. Cada palabra se representa como un vector de dimensión fija, donde la cercanía en el espacio refleja similitud semántica.

Diferencias vs. BoW/TF-IDF: reducción de dimensionalidad, vectores densos que capturan relaciones semánticas y mejor generalización.

GloVe

GloVe (Pennington et al., 2014) crea embeddings a partir de la factorización de una matriz de co-ocurrencia global de palabras. Los vectores resultantes preservan información tanto local (contexto de ventana) como global (distribución en el corpus).

Diferencias vs. Word2Vec: incorpora estadísticas globales del corpus, lo que puede mejorar la representación de palabras infrecuentes; sigue siendo estático.

Embeddings Contextuales

Modelos como BERT (Devlin et al., 2019) generan vectores dinámicos que dependen del contexto completo de la frase. Cada instancia de una palabra recibe un embedding distinto según su uso, capturando matices sintácticos y semánticos más profundos.

Diferencias vs. estáticos: embeddings sensibles al contexto, mayor capacidad para desambiguar significados, a costa de mayor complejidad computacional.

Referencias

- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, y Kristina Toutanova. 2019. «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». En *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 4171-86.
- Jurafsky, Daniel, y James H Martin. 2023. *Speech and Language Processing*. 3.^a ed. <https://web.stanford.edu/~jurafsky/slp3/>.
- Mikolov, Tomas, Kai Chen, Greg Corrado, y Jeffrey Dean. 2013. «Distributed Representations of Words and Phrases and their Compositionality». *arXiv preprint arXiv:1310.4546*.
- Pennington, Jeffrey, Richard Socher, y Christopher D Manning. 2014. «GloVe: Global Vectors for Word Representation». En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532-43.

scikit-learn developers. 2024. «Working with Text Data — scikit-learn 1.4.2 documentation».
https://scikit-learn.org/1.4/tutorial/text_analytics/working_with_text_data.html.